

UC 07 — Codificar back-end de aplicações web

Sistema de Tarefas com Autenticação (MVC)

Curso: Técnico em Informática para Internet
Unidade Curricular: UC 07 — Codificar back-end de aplicações web
Prazo de Entrega: 28/11

Sobre o Projeto

Este projeto de **recomposição de aprendizagem** orienta o aluno a desenvolver um backend completo, consolidando os principais conceitos de desenvolvimento web com Node.js. O sistema implementa autenticação, autorização, consumo de API externa e CRUD de tarefas seguindo o padrão MVC.

Objetivos de Aprendizagem

Ao concluir este projeto, o aluno será capaz de:

- Organizar projetos em **arquitetura MVC**
- Implementar **autenticação com JWT** e autorização baseada em token
- Consumir **APIs externas** (ViaCEP)
- Criar **CRUD completo** com regras de propriedade
- Aplicar **criptografia de senhas** com bcrypt
- Utilizar **boas práticas de versionamento** no GitHub
- Apresentar e explicar código de forma profissional

Tecnologias Utilizadas

Tecnologia	Finalidade
Node.js + Express	Servidor e framework web
ES Modules	type: "module" no package.json
jsonwebtoken (JWT)	Autenticação e autorização
bcrypt	Criptografia de senhas
axios	Consumo da API ViaCEP
uuid	Geração de identificadores únicos
dotenv	Gerenciamento de variáveis de ambiente
nodemon	Reinicialização automática (desenvolvimento)

Estrutura do Projeto

```
projeto-tarefas/  
├── .env  
├── package.json  
├── app.js  
├── src/  
│   ├── controllers/  
│   ├── routes/  
│   ├── models/  
│   ├── middlewares/  
│   └── data/
```

Responsabilidades de Cada Camada

- **app.js**: Configuração do Express, middlewares globais e importação de rotas
- **controllers/**: Lógica de negócio de cada entidade (usuários, tarefas)
- **routes/**: Definição dos endpoints da API
- **models/**: Estrutura e validação dos dados
- **middlewares/**: Autenticação, validações e tratamento de erros
- **data/**: Arrays em memória para armazenamento temporário

Funcionalidades Implementadas

1. Cadastro de Usuários

Endpoint: `POST /auth/registro`

O usuário informa:

- Nome
- Email
- Senha
- CEP

O sistema automaticamente:

- Consulta o **ViaCEP** usando axios
- Completa os campos: rua, bairro, cidade e estado
- Criptografa a senha com **bcrypt**
- Gera um ID único com **uuid**
- Armazena em memória (array)

2. Autenticação com JWT

Endpoint: `POST /auth/login`

- Valida email e senha
- Retorna token JWT com expiração (ex.: 1h)
- Token deve ser enviado no header `Authorization: Bearer <token>`

3. CRUD de Tarefas (Rotas Protegidas)

Todas as rotas de tarefas exigem autenticação via token.

Estrutura da Tarefa

```
{
  id: "uuid-gerado",
  titulo: "Título da tarefa",
  descricao: "Descrição detalhada",
  prazo: "2025-12-01",
  concluida: false,
  userId: "id-do-usuario-proprietario"
}
```

Endpoints

Método	Endpoint	Descrição
GET	/tarefas	Lista tarefas do usuário logado
POST	/tarefas	Cria nova tarefa
PUT	/tarefas/:id	Atualiza tarefa (apenas do proprietário)
DELETE	/tarefas/:id	Remove tarefa (apenas do proprietário)

Regras de Negócio

Cadastro de Usuários

- ☒ **Email único:** Não permitir cadastro com email já existente
- ☒ **Senha mínima:** 6 caracteres
- ☒ **CEP obrigatório:** Consulta ao ViaCEP deve retornar endereço válido
- ☒ **CEP inválido:** Retornar erro claro se a API falhar ou CEP for inválido
- ☒ **Bloqueio de cadastro:** Não salvar usuário sem endereço completo

Autenticação

- ☒ **Token JWT:** Expiração configurável (recomendado: 1h)
- ☒ **Header obrigatório:** `Authorization: Bearer <token>`
- ☒ **Token ausente/inválido:** Retornar código **401 Unauthorized**

Tarefas

- ☒ **Propriedade:** Usuário só pode acessar suas próprias tarefas
- ☒ **Título obrigatório:** Não pode ser vazio
- ☒ **Prazo válido:** Deve ser data no formato ISO
- ☒ **Status inicial:** `concluida: false`

✗ **Acesso não autorizado:** Retornar código **403 Forbidden**

✗ **Tarefa inexistente:** Retornar código **404 Not Found**

Validações e Códigos HTTP

Código	Situação
200	Requisição bem-sucedida
201	Recurso criado com sucesso
400	Dados inválidos (Bad Request)
401	Não autenticado (token ausente/inválido)
403	Não autorizado (tentativa de acesso a recurso de outro usuário)
404	Recurso não encontrado
500	Erro interno do servidor

Persistência

⚠ **Importante:** Os dados são armazenados em arrays na memória e serão **perdidos ao reiniciar o servidor**. Este é um requisito do projeto para focar na lógica de backend sem banco de dados.

Testes e Demonstração

Ferramentas Recomendadas

- **Insomnia** ou **Postman**

Fluxo de Testes Obrigatórios

1. Cadastro de usuário

- Enviar CEP válido
- Verificar preenchimento automático do endereço
- Testar CEP inválido (deve retornar erro)

2. Login

- Fazer login com credenciais corretas
- Copiar o token JWT retornado

3. Autenticação

- Tentar acessar **/tarefas** sem token (deve retornar 401)
- Acessar com token válido (deve funcionar)

4. CRUD de Tarefas

- Criar 2-3 tarefas

- Listar tarefas
- Atualizar uma tarefa
- Marcar tarefa como concluída
- Excluir uma tarefa

5. Autorização

- Criar segundo usuário
 - Tentar acessar/editar tarefa do primeiro usuário (deve retornar 403)
-

Apresentação em Sala

Estrutura da Apresentação (15-20 minutos)

Parte 1: Explicação Teórica (5-7 min)

1. Arquitetura MVC

- Explicar Model, View, Controller
- Como se aplica ao projeto

2. Fluxo de Autenticação

- Registro → Login → Token → Rotas Protegidas
- Demonstrar middleware `autenticar`

3. Organização do Código

- Mostrar estrutura de pastas
- Explicar responsabilidade de cada camada

Parte 2: Demonstração Prática (8-10 min)

4. Demo no Insomnia/Postman

- Executar fluxo completo de testes
- Mostrar respostas da API
- Demonstrar tratamento de erros

5. Walkthrough do Código

- Cadastro + integração ViaCEP
- Geração de token JWT
- Middleware de autenticação
- Controller de tarefas

Parte 3: GitHub e Boas Práticas (3-5 min)

6. Repositório GitHub

- README.md completo

- Histórico de commits limpo
- Instruções de instalação e execução

☑ Critérios de Avaliação

Critério	Peso	Descrição
Funcionalidade	30%	Sistema atende todos os requisitos e regras de negócio
Qualidade do Código	25%	Código limpo, organizado, segue padrões MVC
Documentação	20%	README completo com instruções claras
Apresentação	15%	Domínio do conteúdo e clareza na explicação
GitHub	10%	Commits organizados, estrutura profissional

Checklist de Entrega

- ☐ Repositório público no GitHub
- ☐ README.md com instruções de instalação
- ☐ Arquivo `.env.example` com variáveis necessárias
- ☐ Todos os endpoints funcionando
- ☐ Regras de negócio implementadas
- ☐ Testes manuais validados
- ☐ Código comentado em partes críticas
- ☐ Apresentação preparada

📦 Entrega Final

📅 **Data limite:** 28 de novembro

📢 **Apresentação:** Em horário definido pelo instrutor

Formato de Entrega

1. Link do repositório GitHub no ambiente virtual (Teams)
2. README.md deve incluir:
 - Descrição do projeto
 - Tecnologias utilizadas
 - Como instalar e executar
 - Exemplos de uso (endpoints)
 - Autor e curso

💡 Dicas de Sucesso

- 🔥 **Organize-se:** Divida o projeto em etapas (autenticação → tarefas → testes)
- 🔥 **Commits frequentes:** Faça commits pequenos e descritivos
- 🔥 **Teste constantemente:** Valide cada funcionalidade antes de avançar

- ✦ **Documente:** Comente partes complexas do código
 - ✦ **Prepare a apresentação:** Treine antes do dia da entrega
 - ✦ **Peça feedback:** Valide com colegas antes da entrega final
-



Recursos Adicionais

- [Documentação Express.js](#)
 - [Documentação JWT](#)
 - [API ViaCEP](#)
 - [Guia bcrypt](#)
 - [Boas práticas Node.js](#)
-

Bom trabalho! 🚀